



Are LLMs reliable code reviewers? systematic overcorrection in requirement conformance judgement

Haolin Jin¹ · Huaming Chen¹

Received: 28 February 2026 / Accepted: 14 June 2026 / Published online: 26 June 2026
© The Author(s) 2026

Abstract

Large language models (LLMs) have become essential tools in software development, widely used for requirements engineering, code generation and review tasks. Software engineers often rely on LLMs to verify if code implementation satisfy task requirements, thereby ensuring code robustness and accuracy. However, it remains unclear whether LLMs can reliably determine code against the given task descriptions, which is usually in a form of natural language specifications. In this paper, we uncover a systematic failure of LLMs in matching code to natural language requirements. Specifically, with widely adopted benchmarks and unified prompts design, we demonstrate that LLMs frequently misclassify correct code implementation as non-compliant or defective. Surprisingly, we find that more detailed prompt design, particularly with those requiring explanations and proposed corrections, leads to higher misjudgment rates, highlighting critical reliability issues for LLM-based code assistants. We further analyze the mechanisms driving these failures and evaluate the reliability of rationale-required judgments. Building on these findings, we propose a Fix-guided Verification Filter that treats the model proposed fix as executable counterfactual evidence, and validates the original and revised implementations using benchmark tests and spec-constrained augmented tests. Our results expose previously under-explored limitations in LLM-based code review capabilities, and provide practical guidance for integrating LLM-based reviewers with safeguards in automated review and development pipelines.

Keywords Large language models · LLM-as-a-judge · Over-correction bias · Prompting and verification

✉ Huaming Chen
huaming.chen@sydney.edu.au

Haolin Jin
hjin3177@uni.sydney.edu.au

¹ University of Sydney, Sydney, NSW, Australia

1 Introduction

As large language models (LLMs) have demonstrated increasing capability in the domain of code synthesis (Austin et al. 2021), a growing body of research and tooling efforts have begun exploring their application for automated code review and verification (Xu et al. 2022; Rasheed et al. 2024; Cai et al. 2025). Traditional code reviews require developers to manually verify the alignment between code logic and requirements, a process that is both time consuming and prone to human error. LLMs show significant potential to reduce this burden by automating code assessments and suggesting improvements during the code-review process. For instance, recent studies have leveraged models such as GPT-4o to evaluate code submissions and determine whether they meet quality standards or require revisions (Liu et al. 2023).

In software engineering, verifying and validating that source code aligns with its task requirements remains a challenge (Shankar et al. 2024). Requirements engineering is widely recognized as crucial to clearly defining, understanding, and aligning project objectives with stakeholder needs, thereby reducing risks associated with errors, delays, and project failures (Couder et al. 2024). Recent studies start to investigate the potential of LLMs to bridge this gap. Advanced models such as GPT-4o have shown promising accuracy in identifying unmet requirements from textual descriptions (Reinhold et al. 2024), indicating their potential as reliable “virtual reviewers” even without test cases or formal implementations. To further improve review effectiveness, researchers have explored self-critical mechanisms within LLMs, such as the Self-Refine framework, where models iteratively critique and refine their outputs without additional training (Madaan et al. 2023). Ideally, LLMs should accurately understand functional requirements described in natural language and reliably judge whether provided code satisfies these requirements, assisting developers in identifying defects or confirming correctness. However, in scenarios lacking test cases or reference implementations, the reliability of LLMs in performing such “description-to-code” evaluations remains unclear.

A common intuition is that richer prompting improves reliability: asking the LLM to explain its decision, enumerate mismatches, or propose fixes should encourage more careful reasoning (White et al. 2023). Nevertheless, LLMs are known to exhibit hallucinations and systematic biases (Ji et al. 2023), and prompt “enhancements” may unintentionally shift the model’s error profile rather than uniformly improving accuracy. In particular, more elaborate prompts can amplify an LLM’s tendency to over-criticize correct code or to rationalize incorrect decisions with persuasive-but-unfaithful explanations. This motivates a key question for automated review: *Do prompt variants trade false rejection for false acceptance, and are the generated rationales themselves trustworthy?*

To answer these questions, we investigate whether LLMs can correctly determine code correctness when provided with precise task descriptions and correct implementations. Specifically, we conduct a large-scale empirical investigation across three widely used programming benchmarks under three prompting approaches. We evaluate five representative LLMs (three closed-source and two open-source) and report full confusion-matrix statistics. Beyond decision accuracy, we further examine the reliability of the explanation along two complementary dimensions. Finally, we

proposed a filter-embedded framework to assess their usefulness in reducing such bias. Our experiments reveal a concerning phenomenon: LLMs frequently issue false negative judgments, incorrectly concluding that correct implementations fail to meet stated requirements, resulting in a high false negative rate. Surprisingly, our extended experiments involving various prompt designs indicate that increasing prompt complexity, such as requiring explicit explanations and suggested corrections, counterintuitively leads to higher rates of misjudgment, this finding contradicts the common assumption that incorporating explanatory steps typically enhances the reasoning and accuracy of LLMs (Yu et al. 2023; Chu et al. 2023). Instead, detailed prompts may inadvertently introduce biases toward excessive fault finding, causing models to detect non-existent errors in otherwise correct implementations.

Our findings have direct implications for deploying LLMs as automated reviewers: even when an LLM can frequently “pass” correct solutions, it may simultaneously suffer from severe over-correction (high FN) and unsafe acceptance (high FP), and the accompanying explanations may not faithfully justify the decision. These limitations can undermine the utility and trustworthiness of LLM-based review systems (Collante et al. 2025; Almeida et al. 2024), especially in automated pipelines where a model’s judgement may trigger or block downstream actions (Panickssery et al. 2024), unreliable assessments will compromise the overall effectiveness of automated engineering workflow. Therefore, understanding why LLMs systematically misjudge correct implementations and identifying strategies to mitigate these failures becomes crucial, by providing these novel insights, we aim to raise community awareness regarding the limitations of LLMs in code evaluation. In summary, this work makes the following contributions:

1. **False negatives discovery** - We reveal that LLMs frequently misjudge correct code as failing to meet requirements, indicating their bias towards over-correction rather than accurate verification.
2. **Bias characterization across prompts and five models** - We quantify over-correction bias and false acceptance under three prompting approaches and five LLMs, revealing prompt-induced tradeoffs and decision instabilities.
3. **Self-consistency and Fault-awareness** - We introduce and evaluate an explanation-centric consistency measure that captures contradiction between model verdicts and their rationales, also we evaluate whether LLM rationales reflect fault-aware reasoning, and analyze how false acceptance concentrates on specific bug types.
4. **Exploratory mitigation analysis** - We explore mitigation strategies to assess their effectiveness in reducing judgement biases.

2 Background and related work

2.1 LLMs for code review

Modern code review is a cornerstone of current software development, serving as a primary quality check for defect discovery, maintainability, and knowledge transfer. Empirical work has characterized both the expected benefits and the practical

frictions of review, highlighting that “review quality” depends not only on correctness detection but also on how reviewers interpret requirements and communicate actionable feedback (Bacchelli and Bird 2013; Rigby and Bird 2013; McIntosh et al. 2014). Against this backdrop, learning-based code intelligence has rapidly evolved from task specific neural models to general purpose pre-trained models for code and natural language. Encoder based and encoder-decoder models such as CodeBERT (Feng et al. 2020), GraphCodeBERT (Guo et al. 2020), and CodeT5 (Wang et al. 2021) established strong baselines for code understanding and generation tasks (e.g., summarization, translation, and defect-related reasoning), and provide the representational foundations that later large language models build upon. These capabilities enable an emerging class of LLM-assisted reviewing scenarios, where models are used to summarize changes, propose edits, and explain behavioral implications as part of review.

More recently, code specialized LLMs have demonstrated strong synthesis and editing capabilities, motivating their use as “virtual reviewers” that can generate explanations and suggest repairs. Representative systems and models include Codex style evaluation on HumanEval (Chen et al. 2021), competition-level synthesis with AlphaCode (Li et al. 2022), and open code LLMs such as CodeGen, InCoder, and CodeLlama (Nijkamp et al. 2022; Fried et al. 2022; Roziere et al. 2023). In parallel, software engineering benchmarks targeting real-world issue resolution (e.g., SWE-bench) further illustrate the community’s interest in using LLMs for end-to-end debugging and patching workflows (Jimenez et al. 2023). Our work connects to this trajectory but targets a distinct point in the review pipeline: requirement-conformance judgement without executing tests, and the systematic biases that arise when prompts elicit explanations and fixes.

2.2 Reliability issues in code tasks

A substantial body of work has evaluated LLM capability in code tasks via curated benchmarks with executable tests or reference outputs, such as HumanEval, MBPP, APPS, MultiPL-E, and QuixBugs (Chen et al. 2021; Austin et al. 2021; Hendrycks et al. 2021; Cassano et al. 2022; Ye et al. 2021). These benchmarks have been instrumental in quantifying functional correctness under test-based evaluation and enabling fair comparisons across models. However, they also highlight a practical gap: many real code review settings do not provide comprehensive tests or formal specifications, and “correctness” must be assessed from a natural-language requirement and code alone. In such settings, the key risk is not only incorrect code generation, but also unreliable code assessment, models may reject correct implementations (false rejection) or accept buggy ones (false acceptance) when reasoning is unconstrained by execution.

Reliability concerns in LLM-based reasoning systems have been widely discussed in terms of hallucination, overconfidence, and specification drift, where models produce plausible but unsupported claims (Ji et al. 2023; Liang et al. 2022; Achiam et al. 2023). For code-related tasks, these risks are amplified because models can (i) infer non-existent constraints, (ii) speculate about runtime failures without evidence, or (iii) overfit to stylistic “best practices” that are orthogonal to requirement satisfac-

tion. Moreover, the security community has shown that LLM-generated code can introduce vulnerabilities, which raises the stakes of inaccurate review judgements in practice (Pearce et al. 2025).

Prompting and self-improvement strategies have been proposed to increase reasoning quality, including chain-of-thought prompting, zero-shot “think step-by-step” prompting, and self-consistency sampling (Wei et al. 2022; Kojima et al. 2022; Wang et al. 2022). Iterative refinement and agentic approaches (e.g., Self-Refine and Reflexion) further attempt to improve outputs through critique and feedback cycles (Madaan et al. 2023; Shinn et al. 2023). Related prompting frameworks such as ReAct and prompt-pattern catalogs systematize how to elicit reasoning and tool use (Yao et al. 2023; White et al. 2023). While these approaches can improve performance in many settings, they also introduce new failure modes (e.g., longer rationales that drift from the requirement, or “fixes” that reflect overly conservative assumptions).

2.3 LLMs judge and bias

LLM-based evaluators (“LLM-as-a-judge”) have become a common mechanism for scalable assessment of model outputs, particularly in open-ended generation where reference-based metrics are weak (Gu et al. 2024). Benchmarking frameworks such as MT-Bench and Chatbot Arena operationalize pairwise preference evaluation using strong LLM judges (Chiang et al. 2024), and have been influential in tracking model quality trends (Zheng et al. 2023). More fine-grained LLM-based evaluation protocols, such as G-Eval, propose rubric-style prompting to increase alignment with human judgements (Liu et al. 2023). In code-specific settings, recent work has also explored prompt-based “judge” formulations for code correctness decisions, illustrating the appeal of LLM judges when unit tests are unavailable or incomplete (Tong and Zhang 2024). However, a growing literature warns that LLM judges can be systematically biased and vulnerable. For example, evaluators may exhibit self-preference or other forms of comparative bias, favoring outputs that resemble their own generations rather than outputs that best satisfy the underlying task (Panickssery et al. 2024). More broadly, work on red-teaming language models emphasizes that model behavior can be manipulated through prompt framing and adversarial or leading instructions, which is directly relevant when judgements are elicited via structured prompts that request explanations and fixes (Perez et al. 2022).

3 Methodology

3.1 Research question

To investigate the reliability, bias patterns, and practical implications of LLM-based requirement conformance judgement *without test cases*, we propose the following research questions:

- **RQ1:** Without test cases, to what extent can LLMs reliably assess whether a program conforms to its specification?

- **RQ2:** How does prompt design affect the LLM-based conformance judgement? In particular, does increasing prompt complexity introduce a systematic tradeoff between *false rejection* and *false acceptance*, and how often do decisions flip across prompting modes?
- **RQ3:** What mechanisms drive false acceptance and false rejection? Specifically, (i) are false positives concentrated on particular bug categories (*bug_type* / *failure_symptoms*), and (ii) do false negatives exhibit recurring patterns in the model's rejection rationales that can be summarized by a unified taxonomy?
- **RQ4:** How reliable are the explanations produced under rationale-required prompts? whether the rationale supports the stated YES/NO verdict, and the rationale aligns with known bug categories and failure symptoms?
- **RQ5:** What factors cause LLMs to incorrectly classify correct code as faulty, and can these misjudgments be effectively mitigated?

These research questions cover three key dimensions: understanding the root causes of LLMs' misjudgments, exploring potential mitigation solutions, and evaluating their effectiveness.

3.2 Datasets and construction

To enable reliability analysis of requirement conformance judgement, we require paired data that contains both correct and buggy implementations. We construct a unified benchmark suite based on three widely-used code evaluation benchmarks: HumanEval (Chen et al. 2021), MBPP (Austin et al. 2021), and QuixBugs (Ye et al. 2021). In total, these benchmarks provide over 700 tasks for evaluation; after pairing each task with both a correct and a buggy implementation, the final dataset contains over 1400 instances. For each task, we create two aligned variants sharing the same requirement text: (1) a **canonical** (correct) implementation with label 1, and (2) a **buggy** implementation with label 0. This paired design ensures that false rejection (FN) and false acceptance (FP) can be computed consistently across tasks, prompts, and models, rather than only measuring recognition of correct code.

HumanEval-X-bugs (paired HumanEval) For HumanEval, we adopt HumanEval-X-Bugs (from HumanEvalPack Muennighoff et al. 2024), which provides both a canonical solution and a corresponding buggy solution for each task. Each record includes the natural-language task description and the function declaration, enabling us to construct complete Python implementations. Importantly, HumanEval-X-Bugs additionally provides structured labels describing the injected fault, including *bug_type* and *failure_symptoms*.

Buggy-MBPP construction MBPP (Austin et al. 2021) originally provides task descriptions and correct reference implementations. To support false acceptance analysis, we prepare a paired buggy version of MBPP using an existing bug-aware corpus that provides buggy solution prompts. Since some buggy data is not released as a standalone complete function body, we reconstruct full buggy implementations by combining the buggy prefix/body with the remaining part of the canon-

cal solution, while preserving the same function signature and the overall structure. This yields syntactically complete buggy implementations that are aligned with the original requirements. We further normalize the fault labels into a shared schema so that MBPP, HumanEval-X-Bugs, and QuixBugs can be compared under the same categories.

QuixBugs paired construction QuixBugs (Ye et al. 2021) contains algorithmic problems with known defects and corresponding corrected implementations. We use the corrected version as the canonical implementation and the defective version as the buggy implementation, paired under the same requirement text. Because QuixBugs does not always provide unified fine-grained fault labels in the same form as HumanEval-X-Bugs, we annotate each buggy instance with (i) a normalized `bug_type` and (ii) a `failure_symptoms` label using a standardized labeling protocol, and then map them into the same schema used for the other benchmarks.

Unified schema and label normalization To support consistent aggregation and cross-benchmark analysis, we convert all three datasets into a unified JSON/JSONL schema with fields such as: `task_id`, `requirement` (or `text`), `variant` (canonical/buggy), `label` (1/0), `code`, and (for buggy instances) `bug_type` and `failure_symptoms`. Across datasets, we normalize `bug_type` into a shared set of six coarse categories (e.g., missing logic, excess logic, operator misuse, variable misuse, value misuse, function misuse), and normalize `failure_symptoms` into a compact set describing observable failure modes (e.g., incorrect output, runtime error, non-termination). To facilitate reproducibility, we make the curated datasets and scripts publicly available at <https://github.com/HollinJ3177/Are-LLMs-Reliable-Code-Reviewers-Systematic-Overcorrection-in-Requirement-Conformance-Judgment>.

3.3 Experiment setup

To address RQ1 and RQ2, we designed a series of experiments to evaluate the performance of different LLMs on code requirement conformance tasks and to analyze the impact of prompt design. Regarding model selection, we evaluated five LLMs, including three mainstream closed-source models and two representative open-source models. The closed-source models are GPT-4o, Claude-4.5-sonnet, and Google Gemini-2.0-flash. These models were chosen as representatives of state-of-the-art industry LLMs, recognized for their strong general purpose capabilities in code understanding and generation tasks. Their superior performance makes them promising candidates for automated code review (Rasheed et al. 2024; Fragiadakis et al. 2025; Zhu et al. 2025). However, precisely because of their capabilities, we aimed to examine if they exhibit consistent error patterns under requirement-conformance judgement. Moreover, these models have consistently been among the top choices for code review and code synthesis tasks in previous works (Wang and Chen 2023; Joel et al. 2024; Weysow et al. 2024). To broaden the coverage and improve reproducibility, we further include two open-source instruction-tuned models: `meta-llama/llama-3.1-8b-instruct` and `mistralai/mistral-small-3.1-24b-instruct`.

This setting enables a more comprehensive comparison between closed-source and open-source LLMs under identical evaluation protocols.

Figure 1 summarizes our evaluation pipeline. For each benchmark task, we construct an input package consisting of (i) the natural-language task specification and (ii) two code variants: a canonical reference implementation and a buggy implementation with injected faults, accompanied by the benchmark’s test cases. We then instantiate three prompting modes, for every instance, we record the model outputs in a structured JSON format (including verdict, rationale, and fix when available). Finally, we compute outcome statistics by comparing predicted verdicts against the ground-truth label of each variant (canonical vs. buggy), yielding the confusion matrix and the derived error measures used in our subsequent analyses.

3.4 Prompt design

To ensure a fair comparison, we design three prompting approaches under a unified instruction framework, inspired by recent advances in prompt engineering (Wei et al. 2023; Yao et al. 2023; Jin et al. 2024; Araya 2025; Akbar et al. 2024). Each prompt provides the same input information (the natural language requirement and the code implementation) and differs only in the amount of required intermediate outputs.

Direct prompt The DIRECT prompt asks the model to read the requirement and the provided code implementation, and answer a single question: “Does the code meet the requirement?” The model is instructed to respond with a simple binary answer “Yes” or “No” only. This prompt represents the minimal setting and serves as a baseline.

Direct + explain prompt The DIRECT+EXPLAIN prompt extends the baseline by additionally requesting a short rationale for the judgement. Concretely, the model is first asked to output a binary verdict (“Yes”/“No”), and then provide a brief explanation describing why the code meets or does not meet the requirement. This setting isolates the effect of requiring explicit justification, which is commonly believed to encourage more careful reasoning.

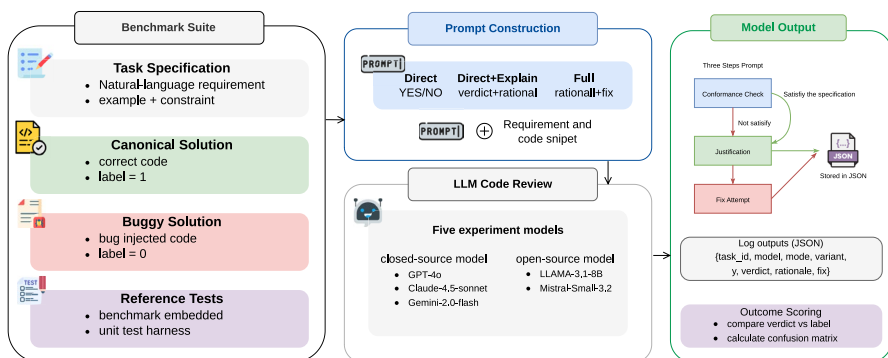


Fig. 1 Workflow of evaluating LLM code conformance on canonical and buggy solutions under three prompting modes, with outputs logged for downstream scoring

Full prompt The FULL prompt further adopts a three-step structure:

- **Judgment:** Read the natural language requirement and the provided code implementation, then answer the question, “Does the code meet the requirement?” with a simple answer “Yes” or “No”.
- **Explanation:** Request the model to provide a rationale for the judgment, such as explaining why it believes the code does not meet the requirements. This covers a detailed analysis of any discrepancies between code logic and requirements.
- **Fix:** If the model judged the code as not meeting the requirement, it was instructed to provide corrected code after the explanation. If the code was deemed correct, this step could be skipped or explicitly noted as unnecessary.

Compared to DIRECT+EXPLAIN, the FULL prompt introduces an explicit repair objective, which allows us to examine whether prompting models to “fix” issues may inadvertently bias them toward over-correction, Fig. 2 presents examples of the Full prompt setting.

Multi-stage prompting strategies have been applied for code evaluation task, aiming to enhance the reasoning performance of LLMs. One common approach first instructs the model to perform a step-by-step analysis of the code’s functionality, and then asks it to summarize the analysis with a binary decision (correct or not). This approach outperforms simple prompts by encouraging LLMs to reason through both the requirements and the code logic before making a judgment (Tong and Zhang 2024). Such findings provide evidence-based support for our design of structured, explanation-driven prompting in code evaluation. Similar two-phase prompting method have also been applied for program repair. For example, Yin et al. (2024); Wang et al. (2025) prompt LLMs to first generate a chain-of-thought diagnosis of

System Prompts

You are a senior software engineer specializing in code review and requirement conformance.

User Prompts

Below is a requirement describing what a Python function is expected to do: {requirement}

And here is the implementation of the function: {code}

Step 1: Conformance Check

Please ignore any indentation error and lack of type hinting! Does the code satisfy the requirement?

Answer only with YES or NO.

Step 2: Justification

If your answer was NO, explain why the code does not meet the requirement.

Be specific and refer to particular lines, conditions, or behaviors.

Step 3: Fix

If your answer was NO, please fix the code which meet the requirement."

Fig. 2 The Full prompt template used in our experiments. It provides the requirement and code, then requests a three-step response

the bug, followed by providing a patch for fixing. Our prompt design is aligned with these approaches, in which the model is initially asked to explain any discrepancies between the code and the requirements, it will use related reasoning to propose a corrected version of the code if the initial answer is negative.

4 Evaluation metrics

To address our research questions, our evaluation considers two complementary dimensions: (i) the correctness of LLM judgements against ground-truth labels, and (ii) the reliability of the generated rationales in explaining and diagnosing failures. To isolate the impact of prompt complexity, we evaluate three progressively more complex prompting strategies: **DIRECT** (judgement only), **DIRECT+EXPLAIN** (judgement with rationale), and **FULL** (three-step prompt consisting of judgement, explanation, and suggested repair), we report all metrics for each benchmark, each model, and each prompt setting.

4.1 Confusion matrix outcomes

With paired canonical/buggy instances (Section 3.2), each instance has a binary ground-truth label $y \in \{1, 0\}$, where $y = 1$ denotes a correct (canonical) implementation and $y = 0$ denotes a buggy implementation. Each model outputs a binary verdict $\hat{y} \in \{1, 0\}$ parsed from “Yes/No” (“Yes” $\rightarrow 1$, “No” $\rightarrow 0$), we compute the following standard outcomes:

$$\begin{aligned} \text{TP} &= \{y = 1 \wedge \hat{y} = 1\}, & \text{FN} &= \{y = 1 \wedge \hat{y} = 0\}, \\ \text{FP} &= \{y = 0 \wedge \hat{y} = 1\}, & \text{TN} &= \{y = 0 \wedge \hat{y} = 0\}. \end{aligned}$$

Intuitively, FN captures *false rejection* (over-correction bias: rejecting correct code), while FP captures *false acceptance* (unsafe acceptance: passing buggy code). To explicitly characterize these two-sided biases, we primarily report the false negative rate measures the over-correction tendency on correct implementations (rejecting correct code) and false positive rate measures unsafe acceptance on buggy implementations (accepting buggy code), both derived from the confusion matrix. In subsequent analyses, we characterize model behavior primarily through these confusion-matrix counts together with FNR/FPR under different prompting modes and benchmarks.

4.2 Rejection reason taxonomy

A key challenge in analyzing false negatives is that, unlike false positives on buggy implementations, FN cases have no ground-truth defect: the implementation is correct (label 1), yet the LLM rejects it (predicts “No”). Therefore, the provided `bug_type` labels (which describe injected faults in buggy code) are not applicable to FN analysis. Instead, FN cases are characterized by the model’s claimed rejection reasons in its rationale, which often reflect systematic over-correction patterns such as requirement hallucination, overemphasis on edge cases, or unsupported assertions.

To systematically study these rejection rationales across benchmarks, prompting modes, and model families, we introduce a unified reason taxonomy. Table 1 summarizes the taxonomy categories used in our study, each FN instance under rationale enabled prompts is mapped to exactly one primary category based on the main reason expressed in the raw rationale, this normalization enables quantitative aggregation and supports further analysis.

The taxonomy serves three purposes in our study. First, it enables us to quantify which rejection patterns dominate FN cases under different prompts (e.g., whether requiring a “Fix” step increases hallucinated requirements or edge-case overconcern). Second, it supports cross-model comparisons to identify whether open-source and closed-source models exhibit different over-correction signatures. Third, it pro-

Table 1 FN reason taxonomy for normalizing the main rationale patterns in those judgments

Category	Explanation
Misread Spec	The model misreads or overlooks key requirement semantics (e.g., interpreting constraint A as constraint B).
Added Requirement	The model introduces constraints not stated in the requirement and rejects the code for violating these hallucinated requirements.
Overthink Edge	The model overemphasizes extreme or unspecified edge cases, despite the requirement not mandating such handling (or the code already covering it).
Assumed Type	The model assumes stricter (or different) input types/formats and rejects the implementation based on those assumptions.
Imagined Runtime	The model speculates about runtime errors (e.g., index error, <code>None</code> handling, exceptions) without concrete evidence from the given code and requirement.
Performance Nitpick	The model treats efficiency/complexity as a hard requirement even when not mentioned, and rejects correct code due to perceived inefficiency.
Readability Nitpick	The model rejects based on style/readability/best-practice concerns rather than functional conformance.
Precision Error	The model over-concerns floating-point precision/rounding and rejects correct implementations without requirement support.
Boundary Error	The model claims off-by-one or boundary-condition errors (e.g., $<$ vs $\leq$) where the implementation is in fact correct.
Logic Error	The model broadly claims incorrect algorithm logic or missing steps, often without a falsifiable counterexample.
Vague Description	The rationale is vague or unsupported, lacking concrete evidence, requirement grounding, or specific failure scenarios.
Other	A catch-all category for unclassifiable patterns.

vides a structured lens for qualitative case studies and for designing bias mitigation explorations, by targeting the most frequent rejection patterns rather than treating all FN cases as homogeneous.

4.3 A1 self-consistency evaluation

While LLMs can output a binary verdict (YES/NO), many prompting approaches additionally request a rationale. However, in practice, the rationale may not faithfully support the final verdict (Turpin et al. 2023; Paul et al. 2024). To quantify this phenomenon, we introduce **self-consistency**, which measures whether an LLM's written rationale aligns with its own stated judgement. A1 is evaluated only for prompting modes that explicitly elicit rationales, i.e., DIRECT+EXPLAIN and FULL. For each evaluated instance, we extract: (1) a normalized verdict $\hat{y} \in \{\text{YES}, \text{NO}\}$ from the model output, and (2) the raw rationale text (the explanation section in the output).

Evaluator and decision criteria We use GPT-4o as an external evaluator to assess whether the rationale supports the stated verdict, the evaluator is instructed *not* to judge the code correctness; instead, it only checks the internal alignment between the verdict and the rationale. For each (verdict, rationale) pair, the evaluator assigns one of three labels: *consistent*, *contradiction*, or *unclear*. We additionally record a contradiction type when contradiction is detected: *NO_but_positive* (verdict is NO but rationale argues the code is correct or compliant), *YES_but_negative* (verdict is YES but rationale argues the code is incorrect or non-compliant), we also ask the model to output their confidence score and short evidence text to support auditing and qualitative inspection.

4.4 A2 fault-awareness evaluation

Beyond internal consistency, a stronger requirement for trustworthy explanations is *fault-awareness*: when the implementation is buggy, does the rationale meaningfully describe the underlying defect and its observable symptoms (Turpin et al. 2023)? To answer this question, we introduce **A2 fault-awareness**, which evaluates whether an LLM rationale aligns with the ground-truth fault labels provided (or normalized) in our paired buggy datasets. A2 is evaluated only on buggy instances (label 0) where ground-truth fault information is available, including a normalized *bug_type* and *failure_symptoms* (Section 3.2), for each buggy instance, the A2 evaluator receives: (1) the model's verdict $\hat{y}$, (2) the raw rationale text, (3) the ground-truth *bug_type*, and (4) the ground-truth *failure_symptoms*. We again use GPT-4o as an external evaluator, instructed to compare the rationale text against the provided ground-truth labels. The evaluator does not decide whether the verdict is correct, it assesses whether the explanation content reflects fault-aware reasoning. For each buggy instance, A2 outputs two alignment judgements: one for *bug_type* and one for *failure_symptoms*, each in: *match*, *mismatch*, *not_mentioned*, or *unclear*. *not_mentioned* indicates the rationale does not discuss the relevant fault dimension at all (e.g., only restating the requirement or giving generic com-

ments), while `unclear` indicates that the rationale mentions a potential issue but the description is too ambiguous to reliably map to the ground-truth label.

5 Results

5.1 Judgement performance and bias

Table 2 reports the false positive rate (FPR) and false negative rate (FNR) of five models across three benchmarks (HumanEval, MBPP, and QuixBugs) under three prompting approaches (DIRECT, DIRECT+EXPLAIN, and FULL). Lower FNR indicates fewer false rejections of correct implementations (i.e., weaker over-correction bias), while lower FPR indicates fewer false acceptances of buggy implementations (i.e., less unsafe acceptance).

Among the three closed-source models, GPT-4o exhibits the most pronounced over-correction pattern as prompt complexity increases. Under DIRECT, GPT-4o achieves a relatively low FNR in HumanEval (26.2%) and MBPP (35.9%), but once explanations and repairs are required, the FNR increases sharply to 73.2% in HumanEval and 87.9% in MBPP. Meanwhile, its FPR drops to nearly zero in several settings (e.g., 0.00% on HumanEval and MBPP under DIRECT+EXPLAIN), suggesting that the model becomes substantially more conservative, rejecting correct code more often while rarely accepting buggy code.

Gemini-2.0-flash shows a comparatively different bias profile: its FNR remains moderate (roughly in the 22.5%-39.6% range across benchmarks and prompts), but

Table 2 False-positive rate (FPR, %) and false-negative rate (FNR, %) of five LLMs on three benchmarks (HumanEval, MBPP, and QuixBugs) under three prompting approaches (DIRECT, DIRECT+EXPLAIN, and FULL). FNR reflects over-correction (rejecting correct implementations), while FPR reflects unsafe acceptance (accepting buggy implementations). Green highlights relatively lower error rates within each model benchmark block, whereas red highlights relatively higher error rates

Model	Approach	HumanEval		MBPP		Quixbugs	
		FPR	FNR	FPR	FNR	FPR	FNR
GPT-4o	Direct	2.44	26.2	3.70	35.9	10.9	35.0
	Direct + Explain	0.00	58.5	0.00	74.1	5.00	45.0
	Full	0.00	73.2	0.20	87.9	5.00	60.0
Gemini-2.0-flash	Direct	8.54	25.6	10.3	34.7	22.5	25.0
	Direct + Explain	7.32	23.2	11.1	35.1	22.5	22.5
	Full	5.49	34.1	7.69	39.6	17.5	32.5
Claude-4-5-sonnet	Direct	2.44	26.2	6.57	58.5	5.00	40.0
	Direct + Explain	1.21	34.1	6.94	55.7	2.50	40.0
	Full	0.61	36.0	5.44	62.3	2.50	50.0
Llama-3.1-8B	Direct	17.1	57.3	3.56	74.7	27.5	52.5
	Direct + Explain	6.71	86.6	0.38	91.9	5.00	87.5
	Full	6.10	84.1	1.88	88.2	30.0	77.5
Mistral-Small-3.1-24B	Direct	6.71	35.9	5.25	60.9	40.0	40.0
	Direct + Explain	14.6	31.1	7.13	47.8	40.0	32.5
	Full	4.88	48.8	4.31	74.3	27.5	62.5

its FPR is consistently higher than GPT-4o and Claude, especially on QuixBugs (22.5% under DIRECT and DIRECT+EXPLAIN). This indicates that Gemini is less over-correcting than GPT-4o under richer prompts, yet more prone to false acceptance on buggy implementations. Claude-4.5 generally achieves low FPR (often $\leq 6\text{--}7\%$) while keeping FNR at a moderate-to-high level depending on the benchmark. Notably, on HumanEval, Claude-4.5 maintains relatively low FNR under DIRECT (26.2%) but increases under DIRECT+EXPLAIN/FULL (36.0%). On MBPP and QuixBugs, FNR remains substantially higher (e.g., 58.5% on MBPP under DIRECT, increasing to 62.3% under FULL), suggesting that for some benchmarks Claude-4.5 also tends toward over-correction, albeit less extremely than GPT-4o.

Answer to RQ1: Across three benchmarks, LLMs are not reliably calibrated for requirement-conformance judgement without executing tests. Table 2 shows substantial false rejection of correct code (high FNR), even under the minimal Direct prompt, and the issue becomes severe under more elaborate prompts. While false acceptance is often lower than FNR, it is non-negligible for some model-benchmark pairs (e.g., higher FPR on QuixBugs for several models), indicating that correctness judgement from specification alone is error-prone and model-dependent.

Open-source models The two open-source models present higher error rates and stronger sensitivity to prompting. Llama-3.1-8B suffers from extremely high FNR across all benchmarks, indicating severe over-correction: for instance, on MBPP, FNR exceeds 74.7% even under DIRECT and rises above 88% under rationale-enabled prompts. QuixBugs also shows high FNR (52.5%–87.5%), implying that the model rejects a large fraction of correct solutions when asked to perform requirement-conformance judgement. Mistral-Small-3.1-24B shows a different failure profile: while its FNR is moderately high across benchmarks, its FPR can be particularly large on QuixBugs (40.0% under DIRECT and DIRECT+EXPLAIN). This suggests that, for certain bug patterns in QuixBugs, the model frequently accepts buggy code as conformant, highlighting a risk of unsafe acceptance.

5.2 Error tradeoff

A key observation from Table 2 is that prompt design does not uniformly improve judgement quality. Instead, increasing prompt complexity often *redistributes* errors between FPR and FNR, forming a clear tradeoff for several models. Intuitively, prompts that ask the model to provide detailed explanations and propose fixes can encourage model’s reasoning, which may reduce false acceptance but simultaneously amplify over-correction bias. For GPT-4o, moving from DIRECT to DIRECT+EXPLAIN and FULL generally decreases FPR while dramatically increasing FNR. The trend is consistent across all three benchmarks, this indicates a strong prompt-induced over-correction effect: richer prompts substantially increase rejection of correct implementations, even though they may also curb false acceptance on buggy ones.

Claude exhibits a qualitatively similar but milder tradeoff. On HumanEval, FPR decreases monotonically as prompts become more complex (2.44% to 0.61%), whereas FNR increases (26.2% to 36.0%). On QuixBugs, the same shift appears (FPR: 5.00% to 2.50%; FNR: 40.0% to 50.0%). These results indicate that increasing

prompt complexity (e.g., explanation- and CoT-style prompting) can indeed elicit deeper auditing behavior, making LLMs more likely to surface potential issues in the code. However, this comes with a clear trade-off: the decision boundary shifts toward conservatism, **reducing false acceptance but amplifying over-correction bias and thus increasing false rejection**. Gemini-2.0-flash shows a weaker strictness shift: FULL reduces FPR across benchmarks (e.g., on HumanEval 8.54% to 5.49%), while FNR increases only moderately from 25.6% to 34.1%.

The open-source models show the strongest non-monotonicity and instability. For Llama-3.1-8B, DIRECT+EXPLAIN sharply increases FNR on all benchmarks (e.g., HumanEval 57.3% to 86.6%; MBPP 74.7% to 91.9%), while FPR may decrease substantially in some settings (e.g., QuixBugs 27.5% to 5.00%). However, adding the repair step in FULL does not consistently improve this trade-off (e.g., on QuixBugs, FPR rebounds to 30.0% while FNR remains high at 77.5%). For Mistral-Small-3.1-24B, DIRECT+EXPLAIN can even move in the opposite direction on HumanEval by increasing FPR (6.71% to 14.6%) while reducing FNR (35.9% to 31.1%).

Implications The tradeoff in Table 2 is not merely a change in percentages, it translates into substantial shifts in the *absolute* number of decisions that developers would have to act on. Figure 3 makes this operational impact explicit. For GPT-4o, richer prompts largely eliminate false acceptance (FP) in some settings (e.g., MBPP FP drops from 19 under DIRECT to 0 under DIRECT+EXPLAIN, staying near-zero under FULL), but the same change simultaneously *more than doubles* false rejection of correct code (FN increases from 184 to 451 on MBPP). A similar pattern appears on HumanEval (FN 43 to 120 while FP 4 to 0). This implies that, in practice, explanation and repair-oriented prompts reduce the small tail of unsafe acceptances, but at the cost of generating a much larger volume of unnecessary rejections and follow-up work, thus increasing the verification burden during the software development.

Answer to RQ2: Prompt design systematically reshapes the error profile rather than uniformly improving judgement quality. As prompts become more detailed, several models shift toward conservative rejection: FPR decreases but FNR increases sharply. This tradeoff is operationally large in absolute terms (Fig. 3), implying frequent decision flips across prompting modes and a substantial increase in unnecessary rejections and follow-up work when explanations or repairs are required.

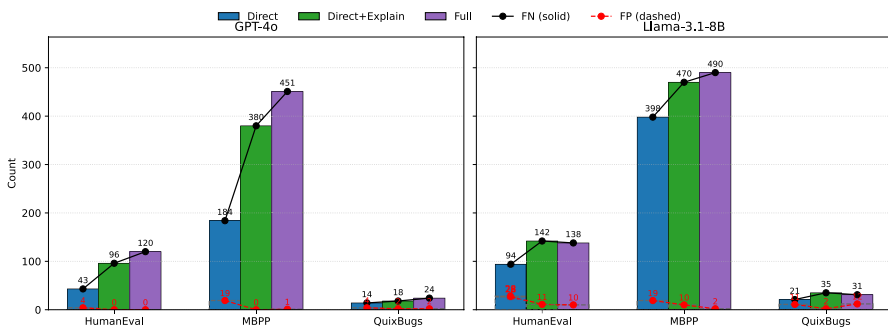


Fig. 3 Absolute FN and FP counts across prompt settings for GPT-4o and Llama-3.1-8B

For the open-source model (Llama-3.1-8B), Fig. 3 suggests an even harsher operational profile: FN dominates across all modes (e.g., MBPP FN 398 to 490), while FP is comparatively smaller and unstable across prompts (MBPP FP 19 to 2). This implies that prompt design should be treated as a *bias control mechanism* rather than a guaranteed accuracy booster: increasing prompt complexity can easily shift the model toward conservative rejection, even when it marginally improves safety against accepting buggy code. Second, the “best” prompt is inherently *cost-sensitive*: scenarios where false acceptance is unacceptable may intentionally adopt stricter prompts and tolerate more FN, while scenarios focused on developer throughput and avoiding unnecessary rework should explicitly control FN and avoid prompts that systematically inflate overcorrection.

5.3 Sensitivity to bug types

FP analysis is grounded on buggy instances where the injected defect labels (*bug_type* and *failure_symptoms*) are available. However, to interpret why models become overly strict (high FNR) or overly permissive (high FPR) under different prompts, it is equally important to examine the *perceived* fault types that models cite in their rationales. In particular, FN cases have no ground-truth defect, so the “bug types” appearing in FN rationales represent the model’s claimed reasons for rejection.

As shown in Fig. 4 left side, nearly half of FN rationales are attributed to *Logic Error* (48.2%, $n = 2018$ out of 4190), where the model broadly claims the algorithm is wrong or “missing steps,” often without a falsifiable counterexample. The next three categories are *Added Requirement* (14.1%, $n = 589$), *Boundary Error* (13.2%, $n = 553$), and *Misread Spec* (11.7%, $n = 492$). Together, these four categories account for 87.2% of all FN explanations, indicating that over-correction is dominated by semantic failure modes rather than superficial critique. In contrast, the quality related categories are comparatively rare: Perf Nitpick (Performance Complexity Nitpick), Read Nitpick (Style Readability Nitpick), and Precision Error (Numeric Precision) jointly contribute a small fraction of FN explanations. This suggests that the dominant over-correction behavior is not simply the model enforcing style, but rather the model functional mis-calibrating by constructing plausible but unsupported failure behaviors.

Figure 4 (right) further breaks down the top-4 perceived errors by model, we observed that the same four categories (*Logic Error*, *Added Requirement*, *Boundary Error*, *Misread Specification*) dominate across all five models, suggesting these are systematic failure modes. For example, Logic Error is the top perceived error for every model (GPT-4o: 470; Claude: 396; Gemini: 220; Llama: 527; Mistral: 405). Second, open-source and closed-source models exhibit different “over-correction signatures.” Llama-3.1-8B shows particularly high counts in Logic Error (527) and Added Requirement (217), implying a stronger tendency to (i) claim algorithmic flaws, and (ii) introduce extra unstated constraints when rejecting correct code. GPT-4o, while also frequently producing Logic Error rationales, shows a comparatively strong presence of Boundary Error claims (169), consistent with a common pattern where the model frames correctness around boundary-case reasoning (e.g., $<$ vs. $\leq$) even when

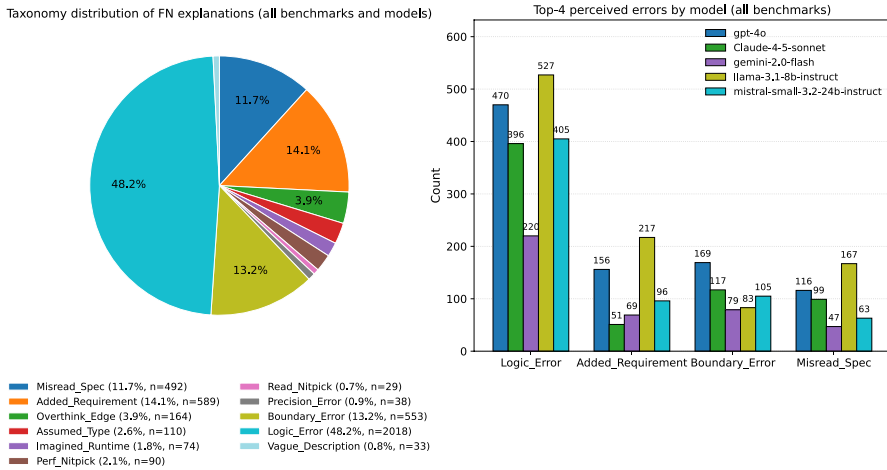


Fig. 4 Distribution of perceived fault types in FN rationales (left) and top-4 perceived errors by model (right). Categories include Misread_Spec (spec misinterpretation), Added_Requirement (unstated constraints), Overthink_Edge (boundary overconcern), Assumed_Type (format assumption), Imagined_Runtime (unsupported runtime speculation), Perf_Nitpick (performance critique), Read_Nitpick (style critique), Precision_Error (numeric precision), Logic_Error (algorithmic flaw claim), and Vague_Description (vague unsupported reasoning)

the implementation is correct. Gemini-2.0-flash has the lowest counts among the five in these categories (e.g., Misread Spec 47), aligning with its comparatively weaker strictness shift observed in the error-tradeoff analysis.

Answer to RQ3: We find recurring and highly concentrated mechanisms behind false rejections. By normalizing FN rationales into a unified taxonomy, four dominant patterns account for 87.2% of FN explanations: Logic Error (48.2%), Added Requirement (14.1%), Boundary Error (13.2%), and Misread Spec (11.7%) (Fig. 4). These results indicate that over-correction is primarily driven by unverified claims and requirement hallucination (inventing unstated constraints), rather than superficial style critique. For buggy implementations, model performance varies strongly by benchmark and model family (Table 2), suggesting that false acceptance is sensitive to the bug patterns and oracle strength of each benchmark (e.g., QuixBugs exhibits higher and more unstable FPR for several models).

These findings suggest that both FP and FN analyses **should not** treat errors as homogeneous. Instead, the most actionable leverage comes from targeting a small set of recurrent “fault narratives.” Specifically, the dominance of Logic Error and Added Requirement indicates that a large portion of rejections are driven by unverified claims. This motivates mitigation strategies that explicitly force evidence grounding, for example, **requiring the rationale to cite the exact requirement clause being violated and to provide a concrete counterexample input/output trace and that discourage unconstrained constraint invention**. Likewise, the prevalence of Boundary Error suggests that models may be over-applying “bug priors” (boundary mistakes are common in real code) as if they were proof of non-conformance, motivating prompts that separate “possible risk” from “confirmed violation.”

6 Explanation reliability

6.1 A1 rationale self-consistency

LLM-based code review tools are typically consumed through explanations rather than binary verdicts alone, therefore, beyond judgment correctness, we evaluate whether the rationale of a model is internally consistent with its own verdict. In A1, we focus on rationale enabled prompting modes (DIRECT+EXPLAIN and FULL) and use a GPT-4o evaluator to label each output as consistent, contradiction, or unclear. We treat contradiction and unclear as *inconsistent rationales*, since both reflect unreliable explanation quality: contradictions explicitly argue against the verdict, while unclear rationales are too vague or mixed to justify the decision.

Inconsistency is non-trivial and benchmark dependent Figure 5 reports the number of inconsistent rationales across the three benchmarks. Overall, inconsistency is not a rare corner case: even strong closed-source models can produce a noticeable amount of contradiction or unclear rationales (Lanham et al. 2023). For instance, GPT-4o shows consistent increases in inconsistency when moving from DIRECT+EXPLAIN to FULL across all benchmarks (HumanEval: 11 → 17, MBPP: 10 → 17, QuixBugs: 3 → 5). Claude-4.5-sonnet exhibits a similar trend, but with a substantially larger jump on MBPP (60 → 89), indicating that longer, repair-oriented prompting can destabilize its own justification under more diverse tasks.

Meanwhile, the effect of adding the repair step is not uniform across models. Gemini-2.0-flash becomes more self-consistent on HumanEval and MBPP (HumanEval: 44 → 23, MBPP: 141 → 97), but becomes less self-consistent on QuixBugs (0 → 8). Open-source models show the clearest dataset sensitivity on QuixBugs: Llama-3.1's inconsistency rises from 4 to 16, and Mistral-Small-3.1 rises from 2 to 18 when switching from Direct+Explain to Full. This is consistent with the intuition that the carefully crafted prompt introduces additional degrees of freedom (repair planning and patch justification), which can amplify explanation instability, especially on small bug-centric benchmarks.

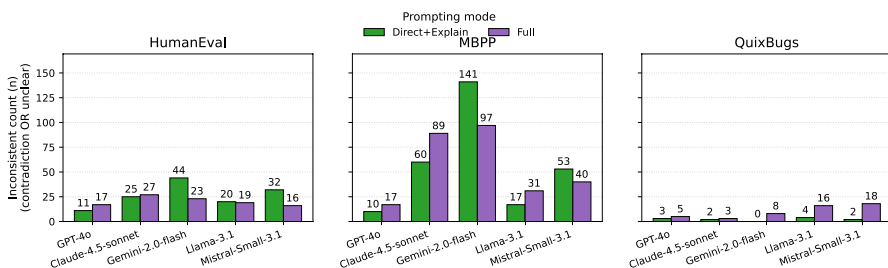


Fig. 5 Counts of inconsistent rationales (labeled as contradiction or unclear by a GPT-4o evaluator) under rationale-enabled prompts. Bars compare Direct+Explain vs. Full for each model across HumanEval, MBPP, and QuixBugs, report absolute counts

Contradictions have model-specific directions Figure 6 further decomposes contradiction cases into two interpretable types: (i) **Contradiction_NO** (verdict=NO, but the rationale is positive), and (ii) **Contradiction_YES** (verdict=YES, but the rationale is negative). We observe a clear directional asymmetry that varies across model families. For GPT-4o, most contradictions are **Contradiction_NO** (93%, $n=61$). In other words, when GPT-4o contradicts itself, it usually rejects the code even though its explanation reads as if the code should be accepted. This suggests that under richer prompts, GPT-4o becomes more conservative in its final verdict, but its explanation does not consistently provide a strong or concrete reason for rejection.

In contrast, Gemini-2.0-flash shows the opposite behavior: 88% of contradictions are **Contradiction_YES** ($n=147$), it outputs “YES” while simultaneously highlighting potential defects. A similar tendency appears in the open-source models (Llama-3.1: 73% **Contradiction_YES**, $n=98$; Mistral-Small-3.1: 72% **Contradiction_YES**, $n=158$). This suggests a pattern that the model preserves an affirmative verdict but produces a rationale that is not aligned with that verdict, which can be particularly misleading in practice because it blurs the meaning of a “YES” judgement (Paul et al. 2024). Claude-4.5-sonnet is comparatively more balanced, implying that its inconsistencies arise from both conservative and permissive failure modes. These results demonstrate that more detailed prompting does not guarantee explanation reliability: richer prompts may elicit longer rationales, but do not ensure that the rationale coherently supports the model’s own verdict. Moreover, the direction of contradiction is model-dependent: GPT-4o tends to reject with supportive rationales, whereas Gemini and the open-source models tend to accept with fault-claiming rationales, practically, this undermines the common assumption that “adding explanations increases trustworthiness.”

6.2 A2 fault-awareness

A2 evaluates whether an LLM’s rationale is fault-aware on buggy instances: when the implementation is incorrect, does the explanation correctly describe (i) the bug type (root cause category) and (ii) the failure symptoms (observable consequence)? Importantly, A2 does not judge whether the verdict itself is correct instead, it checks whether the content of the rationale aligns with the ground-truth fault labels.

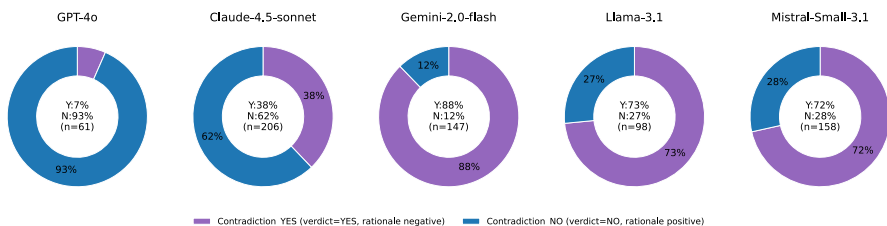


Fig. 6 Directional breakdown of contradiction cases by model. Each chart shows the fraction of Contradiction_NO (verdict=NO but rationale positive) vs. Contradiction_YES (verdict=YES but rationale negative), center labels report percentages and the total number of contradictions (n)

Symptom awareness is high, bug-type awareness is substantially lower Table 3 summarizes A2 match rates under the detailed prompt across three benchmarks. A consistent pattern emerges across all models and datasets: **SymptomMatch is near-ceiling**, while **BugMatch is much lower and varies substantially by benchmark and model**. For example, GPT-4o achieves SymptomMatch of 98.2% (HumanEval), 94.7% (MBPP), and 100.0% (QuixBugs), but its BugMatch is only 59.1%, 70.8%, and 58.3%, respectively, this gap is also visible for the other models. Open-source models show the same qualitative discrepancy: Llama-3.1 has SymptomMatch $\geq 96\%$ across benchmarks, yet BugMatch is only 44.2% (HumanEval), 52.2% (MBPP), and 41.7% (QuixBugs). Taken together, these results indicate that **correct rejection (TN) is frequently driven by symptom-level reasoning** (e.g., “this will produce incorrect output” or “this can fail at runtime”), while **cause-level diagnosis is much less reliable**. Practically, this means that an LLM reviewer can be “right for the inaccurate reason”: it rejects buggy code correctly, but its explanation may still mislead debugging by attributing the failure to an incorrect bug category. Figure 7 shows a concrete example: the rationale correctly identifies the failure symptom (incorrect output), but mis-classifies the underlying cause by framing it as *Missing Logic* even though the ground-truth defect is *Value Misuse*.

Table 3 A2 fault-awareness on buggy instances when the model verdict is correct. We report match rates (%) between the rationale and ground-truth bug type matched and failure symptoms matched

Model	HumanEval		MBPP		QuixBugs	
	BugMatch	SymMatch	BugMatch	SymMatch	BugMatch	SymMatch
GPT-4o	59.1	98.2	70.8	94.7	58.3	100.0
Claude-4.5	65.0	97.5	75.2	93.3	57.9	92.1
Gemini-2.0	52.0	98.7	67.2	95.8	38.2	91.2
Llama-3.1	44.2	96.8	52.2	96.0	41.7	100.0
Mistral-3.1	50.0	96.8	65.7	94.5	58.3	95.8

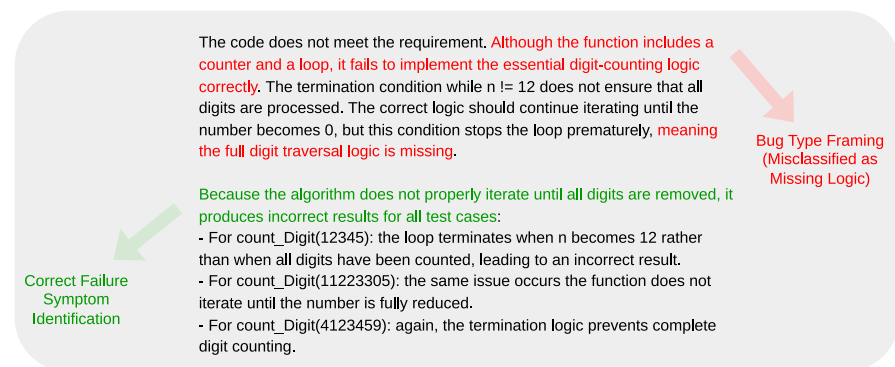


Fig. 7 Example of a symptom is correct but bug type is not matched, the model correctly states the failure symptom, but frames the root cause as Missing Logic (highlighted in red) instead of the ground-truth Value Misuse

Bug-type breakdown To understand why BugMatch is low, we further decompose the bug-type alignment outcomes (match / mismatch / not mentioned / unclear) by ground-truth bug type in Fig. 8 (aggregated over all benchmarks and models under the Full prompt, restricted to TN buggy instances). First, **bug-type match is highly bug-dependent**. *Missing Logic* is the easiest category: it reaches **81% match** with only **17% mismatch**, suggesting that when the defect is framed as an omission (e.g., missing a required check or missing a key step) (Tong and Zhang 2024; Paul et al. 2024), rationales more often converge to the correct causal description. In contrast, *Excess Logic* and *Operator Misuse* show much weaker alignment: *Excess Logic* has 55% mismatch, and *Operator Misuse* is nearly split (46% mismatch/47% match). *Function Misuse* is also unstable, although it has the smallest support ($n = 34$), so conclusions for this category should be treated cautiously. Second, **low BugMatch is more often caused by misdiagnosis than by silence**. Nevertheless, **not mentioned** is not negligible for some bug types: *Operator Misuse* shows the highest not-mentioned proportion (7%), indicating that even in TN cases, a subset of rationales rejects buggy code while failing to explicitly articulate the underlying fault category. This provides direct evidence for our central A2 claim: **verdict correctness does not guarantee fault-aware explanations**.

The A2 results complement our earlier judgement-level bias analysis: while prompt engineering can shift verdict-level behavior, the prompt engineering does not ensure that “more reasoning” yields more diagnostic explanations. Table 3 shows that rationales are generally reliable at stating what goes wrong, but Fig. 8 shows that for several bug classes they are much less reliable at stating why it goes wrong, with mismatch being a prevalent failure mode. This mismatch matters for downstream software engineering workflows: a rationale that correctly flags a defect but attributes it to the wrong cause can still induce ineffective or even harmful repair suggestions, and can waste developer time during debugging (Tong and Zhang 2024). Consequently, A2 suggests that LLM-based code reviewer evaluation should separate deci-

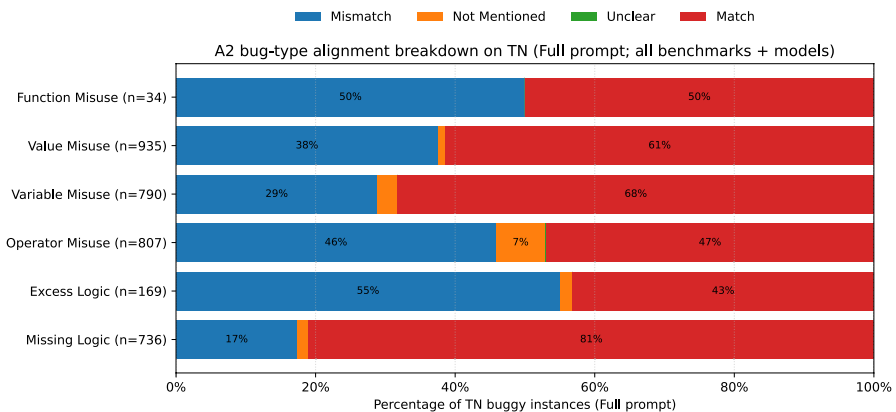


Fig. 8 bug-type alignment on TN buggy instances, Bars report the distribution of rationale labels (match, mismatch, not mentioned, unclear) with respect to ground-truth bug types

sion correctness from explanation fault-awareness, and that mitigation efforts should explicitly target cause-level grounding rather than assuming that longer rationales or repair requests automatically improve diagnosis quality.

Answer to RQ4: *Explanations elicited by rationale required prompts are not consistently reliable. Under A1, we observe non-trivial rates of internally inconsistent rationales (contradiction/unclear), and the contradiction direction is model-dependent. Under A2, rationales are generally strong at describing symptoms but weaker at diagnosing root causes: SymptomMatch is near-ceiling across benchmarks, whereas BugMatch is substantially lower and varies by model (Table 3). Overall, longer explanations do not guarantee faithful justification, and correct verdicts can still be paired with misleading bug diagnoses.*

7 Bias mitigation exploration

7.1 Setup and strategies

Our earlier results reveal a pronounced over-correction bias under Full prompting: when asked to (i) judge conformance, (ii) justify, and (iii) propose a fix, LLMs often become overly conservative and reject correct implementations, leading to a high false-negative rate, while the false-positive rate is comparatively low. This pattern suggests that increasing prompt complexity (e.g., explanation and repair-oriented instructions) can indeed surface more potential issues, but it also shifts the decision boundary toward stricter rejection, amplifying unnecessary “repair” behaviors.

To mitigate this bias, we introduce a lightweight Fix-guided Verification Filter (Fig. 9) that sits inside the evaluation pipeline (rather than post-hoc editing stored predictions). The key intuition is to treat the model’s proposed fix as executable evidence: if a model says NO and proposes a patch, then the pair (*original code*, *fix code*) forms a natural counterfactual for verification. Instead of trusting the textual rationale, we validate both programs with runtime evidence using (1) the benchmark’s reference test suite and (2) a spec-constrained augmented test suite generated by GPT-4o. This design is inspired by the “generate-and-validate” philosophy in test suite based program repair and execution guided verification, where candidate changes are accepted only when they withstand behavioral checks, while also aligning with recent self-refinement or feedback-based LLM frameworks that leverage external signals to correct model behaviors (Madaan et al. 2023; Shinn et al. 2023).

Filter decision cases Given an input program c and a proposed fix $\hat{c}$ (extracted from the response), the filter executes both against the benchmark test cases T and an augmented test set $\tilde{T}$. The final verdict is determined by four common outcomes:

- **Case 1 (equivalent):** c and $\hat{c}$ both pass, and they are behaviorally consistent (same outputs) on spec-constrained augmented inputs $\tilde{T}$. This indicates the fix is unnecessary, we treat the original decision as over-correction and flip the verdict to YES.
- **Case 2 (both fail):** both c and $\hat{c}$ fail on the augmented tests, suggesting either the augmentation over-extended the spec or the repair is still incorrect. We fall

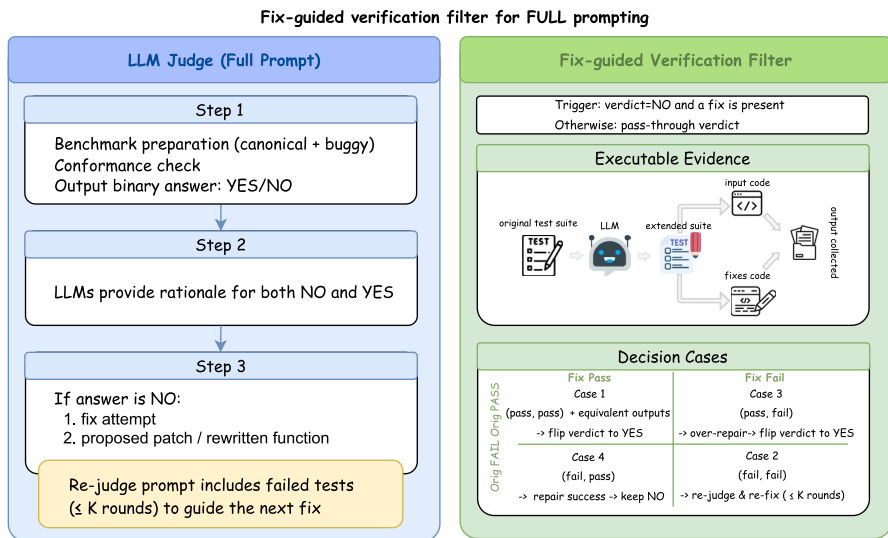


Fig. 9 Overview of the Fix-guided Verification Filter. When the judge outputs NO with a proposed fix, we execute both the original and fixed implementations on the benchmark tests and generated spec-constrained tests, and update the final verdict according to four executable cases

back to the benchmark tests only and allow up to K re-judge rounds, where the next judgment is conditioned on previously failed tests to guide refinement, if the process remains unresolved after K rounds, we keep the original verdict.

- **Case 3 (over-repair):** c passes but $\hat{c}$ fails, meaning the proposed fix introduces regressions, we interpret this as over-repair and flip to YES.
- **Case 4 (repair success):** c fails but $\hat{c}$ passes, indicating the fix corrects genuine nonconformance, we keep the original verdict.

Implementation setup The filter is applied only when the judge returns NO and provides a fix. All five models (GPT-4o, Gemini-2.0-flash, Claude-4.5-sonnet, Llama-3.1-8B, and Mistral-Small-3.1-24B) produce the initial verdict and fix, while the augmented test generation step is standardized to GPT-4o to ensure consistent filtering across judge models. Augmented tests are explicitly constrained by the task specification and examples to avoid introducing adversarial requirements, in our experiments we use $K=2$ rounds, and cache augmented tests to improve reproducibility and robustness.

7.2 Result and analysis

Table 4 reports FPR/FNR before vs. after applying the Fix-guided Verification Filter under Full prompting. Averaged across models, FNR drops from 54.8% to 16.3% on HumanEval and 69.0% to 28.9% on MBPP, while QuixBugs also improves (51.0% to 24.0%). Notably, the gains are strongest for models with extreme baseline conservatism under the Full setup, e.g., Llama-3.1-8B on MBPP (90.81%→23.56%) and GPT-4o on MBPP (88.74%→39.96%), demonstrating that the filter can substantially counteract prompt-induced strictness.

Table 4 False-positive rate (FPR, %) and false-negative rate (FNR, %) before vs. after applying the Fix-guided Verification Filter under FULL prompting. For each FULL+FILTER row, the FNR cell additionally reports the absolute reduction (percentage points) from the corresponding FULL baseline as ↓ (Δ)

Model	Setting	HumanEval		MBPP		QuixBugs	
		FPR	FNR	FPR	FNR	FPR	FNR
GPT-4o	Full (Before)	0.0	70.7	0.0	88.7	5.0	57.5
	Full+Filter (After)	1.8	23.2↓(47.6)	0.4	40.0↓(48.8)	7.5	27.5↓(30.0)
Gemini-2.0-flash	Full (Before)	7.8	25.0	11.1	30.1	22.5	20.0
	Full+Filter (After)	6.4	5.8↓(19.2)	10.2	10.2↓(19.9)	22.5	12.5↓(7.5)
Claude-4.5-sonnet	Full (Before)	1.9	46.0	5.3	61.2	5.0	50.0
	Full+Filter (After)	3.1	8.7↓(37.3)	5.6	27.2↓(34.0)	7.5	22.5↓(27.5)
Llama-3.1-8B	Full (Before)	4.9	82.3	1.5	90.8	17.5	70.0
	Full+Filter (After)	6.1	27.4↓(54.9)	1.7	23.6↓(67.3)	20.0	25.0↓(45.0)
Mistral-Small-3.2-24B	Full (Before)	3.7	50.0	4.3	74.1	22.5	57.5
	Full+Filter (After)	5.5	16.5↓(33.5)	4.7	43.5↓(30.6)	25.0	32.5↓(25.0)

The observed improvements are aligned with the mechanism of the filter: when an LLM proposes a fix for a correct program, that fix often does not introduce new behavior, by requiring the original and fixed code to be consistent under executable tests (including spec-constrained augmentation), the filter rejects spurious NO decisions and recovers correct YES outcomes. Conceptually, this resembles “execution as an arbiter” and “patch validation” in test-driven repair, where runtime evidence is used to gate model-driven changes and avoid unnecessary edits. In our setting, the same idea serves as a bias mitigation tool: it converts the model’s own patch into a verifiable hypothesis and uses differential evidence to decide whether the initial rejection was warranted. While the filter is designed primarily to mitigate over-correction bias, FPR can increase slightly after filtering (e.g., GPT-4o on HumanEval: 0.00%→1.83%; several models on QuixBugs: +2.5 points). This is expected: the filter is triggered only on NO+fix cases, and flipping some of these rejections to YES can inadvertently accept a small number of buggy programs that pass limited tests. This effect is most visible on QuixBugs, where the reference test suites are often shallow, reducing the discriminative power of execution-based checks; even spec-constrained augmentation cannot fully compensate for missing oracle coverage. Practically, this suggests the filter is most reliable when the benchmark harness provides reasonable behavioral coverage (as in HumanEval/MBPP), and it motivates future extensions such as stronger augmentation (e.g., metamorphic relations or property-based generators), multi-oracle agreement, or combining execution evidence with lightweight static checks to further limit residual false acceptances. Overall, Table 4 shows that the Fix-guided Verification Filter provides a simple and model-agnostic way to substantially reduce over-correction bias under Full prompting, by grounding verdict revision in executable evidence rather than purely textual rationales.

Answer to RQ5: False rejection of correct code is primarily caused by prompt-induced conservatism and rationale-driven overreach: models frequently hallucinate extra constraints, over-emphasize edge cases, or assert vague “logic errors” without executable evidence (Fig. 4). To mitigate this over-correction bias, we introduce a Fix-guided Verification Filter that treats the model-proposed fix as an executable counterfactual and validates under benchmark and spec-constrained augmented tests (Fig. 9). The filter substantially reduces FNR across all five models and three benchmarks, with only modest FPR increases in some settings, this suggests misjudgments can be mitigated by grounding decisions in executable evidence rather than relying solely on increasingly elaborate prompts.

8 Threats to validity

This section discusses limitations and potential confounding factors in our experimental design and the proposed Fix-guided Verification Filter, organized following common validity dimensions.

8.1 Construct validity

A central construct in our study is over-correction (rejecting correct implementations), which is closely related to but not identical to LLMs hallucination (Ji et al. 2023). Many false rejections are accompanied by confident rationales that introduce unstated constraints or speculative failure scenarios, which resembles requirement hallucination in natural-language generation. However, our current analysis does not explicitly align the taxonomy of FN rationales with established hallucination definitions and measurements (e.g., intrinsic vs. extrinsic hallucination) (Alansari and Luqman 2025). Consequently, some of the observed misjudgments may be partially explained by broader hallucination behaviors rather than a unique code-review specific bias. Future work could bridge this gap by jointly evaluating FN rationales with hallucination-oriented metrics and annotator guidelines, and by explicitly modeling whether each rejection claim is supported by the given specification.

Operationalization of correctness via benchmark labels and tests We treat canonical solutions as correct (label = 1) and buggy solutions as incorrect (label = 0), and compute FNR/FPR accordingly. While this paired design enables controlled comparisons, it assumes that canonical solutions and embedded tests reflect the intended specification. In practice, benchmark tests can be incomplete, and canonical solutions may encode one of several valid behaviors consistent with the natural-language requirement. As a result, a model may be penalized for rejecting a canonical implementation due to an alternative (but still plausible) spec interpretation, or a buggy implementation may slip through weak tests and appear acceptable. This is particularly salient for small suites (e.g., QuixBugs), where limited oracle coverage can blur the boundary between functional non-conformance and under-specification.

Prompt-mode construct and interpretation Our three prompting modes (DIRECT, DIRECT+EXPLAIN, FULL) are meant to represent progressively richer reviewer instructions. Still, prompt semantics can implicitly signal different review criteria (e.g., encouraging best-practice critique or performance concerns) even when not required by the task (Zhu et al. 2023). Therefore, part of the measured “bias shift” may reflect a change in the latent evaluation rubric induced by the prompt, rather than a pure reasoning-quality change.

8.2 Internal validity

Sensitivity to prompt wording and formatting Results can change with prompt phrasing. Although we use common, widely adopted prompt patterns, minor changes (e.g., stricter instruction to ignore efficiency, different formatting of requirements/code

blocks, or alternative delimiters) can alter the model's decision boundary and thus the measured FNR/FPR. This introduces a threat that some effects may be prompt-instance specific, a stronger design would include multiple paraphrased prompts per mode and report variance (or worst/best-case bounds) across prompt templates.

Non-determinism and API/runtime variability Even with temperature = 0, model-serving infrastructure can introduce slight non-determinism (e.g., backend updates, routing, transient failures). Additionally, tool-chain details (Python version, timeouts, process isolation) may affect whether executions terminate or time out, which can influence filter decisions. We partially mitigate this by logging raw outputs, caching augmented tests, and using consistent execution budgets, but residual nondeterminism may still affect edge cases.

Verification is only as strong as the tests The Fix-guided Verification Filter relies on the benchmark test suite plus GPT-generated spec-constrained augmentations. If the augmented tests fail to cover the relevant behavioral space, the filter may incorrectly treat two implementations as behaviorally equivalent (Case 1) or accept a buggy implementation that still passes the available tests. This is a key reason why mitigation can reduce FNR substantially while sometimes slightly increasing FPR: flipping some NO to YES decisions can inadvertently pass buggy code under insufficient test coverage, this is an inherent limitation of execution-based validation without a formal oracle.

Evaluator coupling and potential judge bias in augmentation Although the initial verdicts and fixes come from five different judge models, the augmentation step is standardized to GPT-4o. This improves consistency between conditions, but introduces dependency on a single model's prior testing. If GPT-4o systematically under-generates certain edge cases, the filter may overestimate equivalence. Conversely, if it over-generates constraints, the filter may become overly strict. A more robust alternative would triangulate augmentation using multiple generators or incorporate metamorphic relations or property-based testing.

8.3 External validity

Generalizability beyond benchmark Python functions HumanEval, MBPP, and QuixBugs contain relatively small, self-contained functions with short specifications and unit-test harnesses. Real-world code review often involves larger codebases, hidden dependencies, stateful APIs, and non-functional requirements (performance, security, maintainability), therefore, the measured bias patterns and the effectiveness of the filter may not directly transfer to industrial settings.

Model coverage and version drift We evaluate five representative LLMs (three closed-source and two open-source). Closed-source models evolve rapidly, so measured rates may change as providers update models. For open-source models, alternative instruction-tuned variants or larger parameter scales may exhibit different bias

profiles. While we report results for a diverse set of models, the findings should be interpreted as evidence about these specific model snapshots rather than immutable properties of the families.

9 Conclusion

In this paper, we revisit whether a model can reliably judge requirement conformance from a natural-language specification and an implementation without executing tests. Through a unified evaluation on three widely used benchmarks and five representative LLMs, we uncover a systematic reliability gap: models frequently reject correct implementations, and this over-correction tendency can become substantially worse as prompts are enriched with explanation and repair requirements. Our results demonstrate that prompt “enhancements” often act as a decision-boundary control rather than a guaranteed accuracy booster. In particular, explanation and fix-oriented prompting can sharply increase false rejections while reducing false acceptances, producing a pronounced trade-off that also translates into large operational costs in absolute FN/FP counts. Beyond verdict accuracy, we show that rationales are not consistently trustworthy: models can produce internally inconsistent explanations and exhibit high symptom-level but weaker cause-level fault awareness, implying that persuasive rationales may still be unfaithful or diagnostically misleading. To mitigate over-correction bias, we propose a Fix-guided Verification Filter that treats the model’s proposed fix as executable counterfactual evidence and validates the original vs. fix behavior in augmented tests with restrictions on parameters and specifications. This lightweight framework reduces false-negative rates across all tested models and benchmarks, with only modest increases in false-positive rates in some settings.

Author Contributions Hollin Jin: Conceptualization, Methodology, Dataset Construction, Software Implementation, Formal Analysis, Visualization, Writing – Original Draft, Writing – Review & Editing. Huaming Chen: Supervision, Conceptualization, Writing – Review & Editing. All authors have read and approved the final manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions.

Data Availability The curated paired datasets and preprocessing scripts are publicly available in our GitHub repository <https://github.com/HollinJ3177/Are-LLMs-Reliable-Code-Reviewers-Systematic-Overcorrection-in-Requirement-Conformance-Judgement>.

Code Availability The experimental framework and mitigation filter implementation are available in the same repository.

Declarations

Competing Interests The authors declare no competing interests.

Ethics Approval and Consent to Participate Not applicable as this study did not involve human participants or animals.

Consent for Publication Not applicable as this study does not contain any individual person’s data.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S.: Gpt-4 technical report. [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) (2023)
- Akbar, S.A., Hossain, M.M., Wood, T., Chin, S.-C., Salinas, E.M., Alvarez, V., Cornejo, E.: Hallumeasure: Fine-grained hallucination measurement using chain-of-thought reasoning. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 15020–15037 (2024)
- Alansari, A., Luqman, H.: Large Language Models Hallucination: A Comprehensive Survey. [arxiv:2510.06265](https://arxiv.org/abs/2510.06265) (2025)
- Almeida, Y., Albuquerque, D., Dantas Filho, E., Muniz, F., Farias Santos, K., Perkusich, M., Almeida, H., Perkusich, A.: Aicodereview: Advancing code quality with ai-enhanced reviews. *SoftwareX* **26**, 101677 (2024)
- Araya, R.: Do Chains-of-Thoughts of Large Language Models Suffer from Hallucinations, Cognitive Biases, or Phobias in Bayesian Reasoning?. [arxiv:2503.15268](https://arxiv.org/abs/2503.15268) (2025)
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C.: Program Synthesis with Large Language Models. [arxiv:2108.07732](https://arxiv.org/abs/2108.07732) (2021)
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q.: Program synthesis with large language models. [arXiv:2108.07732](https://arxiv.org/abs/2108.07732) (2021)
- Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 35th International Conference on Software Engineering (ICSE), pp. 712–721. IEEE (2013)
- Cai, Y., Hou, Z., Sanan, D., Luan, X., Lin, Y., Sun, J., Dong, J.S.: Automated program refinement: Guide and verify code large language model with refinement calculus. *Proceed. ACM Program. Lang.* **9**(POPL), 2057–2089 (2025)
- Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C.J., Feldman, M.Q., et al.: Multipl-e: A scalable and extensible approach to benchmarking neural code generation. [arXiv:2208.08227](https://arxiv.org/abs/2208.08227) (2022)
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G.: Evaluating large language models trained on code. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374) (2021)
- Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A.N., Li, T., Li, D., Zhu, B., Zhang, H., Jordan, M., Gonzalez, J.E., et al: Chatbot arena: An open platform for evaluating llms by human preference. In: Forty-first International Conference on Machine Learning (2024)
- Chu, Z., Chen, J., Chen, Q., Yu, W., He, T., Wang, H., Peng, W., Liu, M., Qin, B., Liu, T.: A survey of chain of thought reasoning: Advances, frontiers and future. **1**. [arXiv:2309.15402](https://arxiv.org/abs/2309.15402) (2023)
- Collante, A., Abedu, S., Khatoonabadi, S., Abdellatif, A., Alor, E., Shihab, E.: The impact of large language models (llms) on code review process. [arXiv:2508.11034](https://arxiv.org/abs/2508.11034) (2025)
- Couder, J.O., Gomez, D., Ochoa, O.: Requirements verification through the analysis of source code by large language models. In: SoutheastCon 2024, pp. 75–80. IEEE (2024)
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al: Codebert: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547 (2020)
- Fragiadakis, G., Diou, C., Kousiouris, G., Nikolaidou, M.: Evaluating Human-AI Collaboration: A Review and Methodological Framework. [arxiv:2407.19098](https://arxiv.org/abs/2407.19098) (2025)
- Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-T., Zettlemoyer, L., Lewis, M.: InCoder: A generative model for code infilling and synthesis. [arXiv:2204.05999](https://arxiv.org/abs/2204.05999) (2022)
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al.: A survey on llm-as-a-judge. *The Innovation* (2024)

- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S.: Graph-codebert: Pre-training code representations with data flow. [arXiv:2009.08366](#) (2020)
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D.: Measuring coding challenge competence with apps. [arXiv:2105.09938](#) (2021)
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**(12), 1–38 (2023)
- Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: Swe-bench: Can language models resolve real-world github issues? [arXiv:2310.06770](#) (2023)
- Jin, B., Xie, C., Zhang, J., Roy, K.K., Zhang, Y., Li, Z., Li, R., Tang, X., Wang, S., Meng, Y., Han, J.: Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs. [arxiv:2404.07103](#) (2024)
- Joel, S., Wu, J.J., Fard, F.H.: A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. [arxiv:2410.03981](#) (2024)
- Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. *Adv. Neural. Inf. Process. Syst.* **35**, 22199–22213 (2022)
- Lanham, T., Chen, A., Radhakrishnan, A., Steiner, B., Denison, C., Hernandez, D., Li, D., Durmus, E., Hubinger, E., Kernion, J.: Measuring faithfulness in chain-of-thought reasoning. [arXiv:2307.13702](#) (2023)
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al.: Competition-level code generation with alphacode. *Science* **378**(6624), 1092–1097 (2022)
- Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A.: Holistic evaluation of language models. [arXiv:2211.09110](#) (2022)
- Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Adv. Neural. Inf. Process. Syst.* **36**, 21558–21572 (2023)
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-Refine: Iterative Refinement with Self-Feedback. [arxiv:2303.17651](#) (2023)
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. *Adv. Neural. Inf. Process. Syst.* **36**, 46534–46594 (2023)
- McIntosh, S., Kamei, Y., Adams, B., Hassan, A.E.: The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects. In: *Proceedings of the 11th Working Conference on Mining Software Repositories*, pp. 192–201 (2014)
- Muennighoff, N., Liu, Q., Zebaze, A., Zheng, Q., Hui, B., Zhuo, T.Y., Singh, S., Tang, X., Werra, L., Longpre, S.: OctoPack: Instruction Tuning Code Large Language Models. [arxiv:2308.07124](#) (2024)
- Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: Codegen: An open large language model for code with multi-turn program synthesis. [arXiv:2203.13474](#) (2022)
- Panickssery, A., Bowman, S., Feng, S.: Llm evaluators recognize and favor their own generations. *Adv. Neural. Inf. Process. Syst.* **37**, 68772–68802 (2024)
- Paul, D., West, R., Bosselut, A., Faltings, B.: Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. In: Al-Onaizan, Y., Bansal, M., Chen, Y.-N. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 15012–15032. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.882>. <https://aclanthology.org/2024.findings-emnlp.882/>
- Paul, D., West, R., Bosselut, A., Faltings, B.: Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 15012–15032 (2024)
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* **68**(2), 96–105 (2025)
- Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., Irving, G.: Red teaming language models with language models. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 3419–3448 (2022)
- Rasheed, Z., Sami, M.A., Waseem, M., Kemell, K.-K., Wang, X., Nguyen, A., Systä, K., Abrahamsson, P.: Ai-powered code review with llms: Early results. [arXiv:2404.18496](#) (2024)
- Reinhold, L.M., Schieseck, M., Wagner, L.P., Gehlhoff, F., Fay, A.: Exploring llms for verifying technical system specifications against requirements. [arXiv:2411.11582](#) (2024)

- Rigby, P.C., Bird, C.: Convergent contemporary software peer review practices. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, pp. 202–212. (2013)
- Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T.: Code llama: Open foundation models for code. [arXiv:2308.12950](https://arxiv.org/abs/2308.12950) (2023)
- Shankar, S., Zamfirescu-Pereira, J., Hartmann, B., Parameswaran, A., Arawjo, I.: Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In: Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, pp. 1–14. (2024)
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. *Adv. Neural. Inf. Process. Syst.* **36**, 8634–8652 (2023)
- Tong, W., Zhang, T.: Codejudge: Evaluating code generation with large language models. [arXiv:2410.02184](https://arxiv.org/abs/2410.02184) (2024)
- Turpin, M., Michael, J., Perez, E., Bowman, S.: Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Adv. Neural. Inf. Process. Syst.* **36**, 74952–74965 (2023)
- Wang, J., Chen, Y.: A review on code generation with llms: Application and evaluation. In: 2023 IEEE International Conference on Medical Artificial Intelligence (MedAI), pp. 284–289 (2023). <https://doi.org/10.1109/MedAI59581.2023.00044>
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. [arXiv:2203.11171](https://arxiv.org/abs/2203.11171) (2022)
- Wang, Y., Wang, W., Joty, S., Hoi, S.C.: Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 8696–8708 (2021)
- Wang, Y., Ji, P., Yang, C., Li, K., Hu, M., Li, J., Sartoretti, G.: Mcts-judge: Test-time scaling in llm-as-a-judge for code correctness evaluation. [arXiv:2502.12468](https://arxiv.org/abs/2502.12468) (2025)
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. [arxiv:2201.11903](https://arxiv.org/abs/2201.11903) (2023)
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural. Inf. Process. Syst.* **35**, 24824–24837 (2022)
- Weyssow, M., Kamanda, A., Zhou, X., Sahraoui, H.: CodeUltraFeedback: An LLM-as-a-Judge Dataset for Aligning Large Language Models to Coding Preferences. [arxiv:2403.09032](https://arxiv.org/abs/2403.09032) (2024)
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with chatgpt. [arXiv:2302.11382](https://arxiv.org/abs/2302.11382) (2023)
- Xu, F.F., Alon, U., Neubig, G., Hellendoorn, V.J.: A systematic evaluation of large language models of code. In: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, pp. 1–10. (2022)
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. [arxiv:2210.03629](https://arxiv.org/abs/2210.03629) (2023)
- Ye, H., Martinez, M., Durieux, T., Monperrus, M.: A comprehensive study of automatic program repair on the quixbugs benchmark. *J. Syst. Softw.* **171**, 110825 (2021)
- Yin, X., Ni, C., Wang, S., Li, Z., Zeng, L., Yang, X.: Thinkrepair: Self-directed automated program repair. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1274–1286 (2024)
- Yu, Z., He, L., Wu, Z., Dai, X., Chen, J.: Towards better chain-of-thought prompting strategies: A survey. [arXiv:2310.04959](https://arxiv.org/abs/2310.04959) (2023)
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al.: Judging llm-as-a-judge with mt-bench and chatbot arena. *Adv. Neural. Inf. Process. Syst.* **36**, 46595–46623 (2023)
- Zhu, L., Wang, X., Wang, X.: JudgeLM: Fine-tuned Large Language Models are Scalable Judges. [arxiv:2310.17631](https://arxiv.org/abs/2310.17631) (2025)
- Zhu, K., Wang, J., Zhou, J., Wang, Z., Chen, H., Wang, Y., Yang, L., Ye, W., Zhang, Y., Gong, N., et al.: Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In: Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis, pp. 57–68 (2023)